Process & Decision Documentation

This document is used to make your design and development process visible. At this stage of your academic career, you are expected not only to produce finished work, but to articulate how decisions were made, how ideas changed, and how collaboration (for the assignments that include group work) unfolds

In professional and co-op contexts, employers do not only evaluate your final projects in your portfolio. They often ask candidates to explain their process, justify trade-offs, reflect on iteration, and describe their roles within a team.

You will need to submit a modification of this document for every group assignment (A1 – A3) and a shorter version for your individual assignments (Side Quests and A4).

For A1 – A3, this is a group document submitted once per group. Each group member must clearly document their own role and responsibilities. Different roles will naturally produce different design processes.

Side Quests and A4 (Individual Work)

Keep this section brief, typically 2 to 4 sentences.

Focus on:

- One significant decision or change you made
- Why you made it
- What effect it had on the work

Examples:

- Simplifying a mechanic so it functioned correctly
- Changing an approach after something failed
- Deciding not to pursue an idea due to time or technical limitations

You are not expected to document every alternative or iteration

Entry Header

Name: Ria Anand

Role(s): Designer, Programmer

Primary responsibility for this work: Coding a two level grid-based puzzle game.

Goal of Work Session

- Make a playable maze puzzle with two levels and make sure that all game elements work.
- Debugging code

Tools, Resources, or Inputs Used

- GenAI tool: ChatGPT 5.2
- Prior code drafts of the maze and level class
- Manual playtesting within the p5.js sketch

GenAI Documentation

Everyone must complete this section. If not GenAI was used, write, "No GenAI used for this task." When GenAI is not used, process evidence should still demonstrate iteration, revision, or development over time.

Because GenAI can closely mimic human-created work, instructors or TAs may occasionally request additional process evidence to confirm non-use. This may include original working files (e.g., an illustrator file), intermediate drafts, or a brief check-in with a TA to walk through your process.

These requests are not an assumption of misconduct. They are part of ensuring academic integrity in an environment where distinguishing between human-created and AI-generated work is increasingly difficult.

If GenAI was used (keep each response as brief as possible):

Date Used: February 2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use:

- Debugging and restructuring the game code

Summary of Interaction:

- Helped rewrite code based on example from class
- Helped rewrite the code to prevent blocked circles in the game

- Assisted in iterations of code and debugging

Human Decision Point(s):

- Replaced the entire code multiple times because I was not satisfied with the AI output
- Chose the circle spawn points per level for clarity

Integrity & Verification Note:

- Reviewed all generated code line by line
- Play tested both levels of code to confirm all circles were reachable
- Double-checked alignment with course content

Scope of GenAI Use:

- Did not design maze layout or overall game concept
- Did not write entire code, modified code from class

Limitations or Misfires:

- Earlier code iterations were not accurate and code needed to be rewritten multiple times

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome. This may include:

- Started from maze example
- Added player movement and game elements
- Discovered unreachable circles through playtesting
- Multiple iterations of code
- Reworked the level structure for cleaner layout
- Re-tested after each change and end code to confirm functionality

Decision Points & Trade-offs

Describe one or two key decisions you made:

- Chose to use AI for code iterations
- Regenerated code instead of over editing to prevent over errors
- Prioritized clarity of game rather than adding advanced features

Verification & Judgement

Explain how you evaluated whether your decision or change was appropriate:

- Played through both levels multiple times
- Confirmed:
    - All circles were reachable
    - Walls block movements correctly
    - Level progression and win state at end

Limitations, Dead Ends, or Open Questions

Note anything that:

- Partial fixes were needed to collect the circles and broke the code a few times
- Other features were not added to the game because I wanted to have simple functioning code rather than a complicated game that does not work.

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.

Note: If GenAI is used in another language, you must include both the original language and a translated transcript in the appendix

Ria:

Add a playable puzzle game to the code below of a maze and make the games have two levels

/*

Week 4 — Example 2: Level class (hard-coded grid)

Goal of this example:

- Keep the exact same *visual output* as Example 1 (a drawn maze).

- Change the *structure* of the code so that “a level” becomes a reusable object.

Key teaching idea:

- A class lets you bundle data (the grid) + behavior (draw, size helpers)
  into one thing you can reuse later (especially once you start loading
  multiple levels from JSON).

*/

// ----------------------------

// 1) Global constants

// ----------------------------

// Tile Size (TS) = how many pixels wide/tall each grid cell is.

// If TS=32, then each number in the 2D grid draws as a 32x32 square.

const TS = 32;

// ----------------------------

// 2) Level class definition

// ----------------------------

/*

A Level is responsible for:

- storing the grid (2D array of numbers)

```
- knowing how big it is (rows/cols and pixel dimensions)
- drawing itself

Important concept:
- The sketch (setup/draw) should not need to know the details
  of how the level draws—just that it can draw.
*/
class Level {
  /*
  The constructor runs when you do: new Level(grid, TS)

  Parameters:
  - grid: a 2D array like grid[row][col]
  - tileSize: TS (pixels per tile)
  */
  constructor(grid, tileSize) {
    this.grid = grid;
    this.ts = tileSize;
  }

  // How many columns are in the grid?
  // (We assume each row has the same number of columns.)
  cols() {
```

```
  return this.grid[0].length;
}

// How many rows are in the grid?
rows() {
  return this.grid.length;
}

// Pixel width of the level = columns * tileSize
pixelWidth() {
  return this.cols() * this.ts;
}

// Pixel height of the level = rows * tileSize
pixelHeight() {
  return this.rows() * this.ts;
}

/*
Draw the whole grid.

Tile legend (same as Example 1):
- 0 = floor
```

- 1 = wall

This method contains the “nested for loop” logic, so the sketch doesn’t have to.

Later, you can extend this to:

- draw special tiles

- draw decorations

- expose collision checks (isWall, etc.)

*/

```
draw() {
  // Loop over each row...
  for (let r = 0; r < this.rows(); r++) {
    // ...and each column in that row.
    for (let c = 0; c < this.cols(); c++) {
      const tileValue = this.grid[r][c];

      // Choose a colour based on the tile.
      if (tileValue === 1) {
        fill(30, 50, 60); // wall colour (dark teal)
      } else {
        fill(230); // floor colour (light gray)
      }

      // Convert grid coordinates (row/col) into pixel coordinates (x/y).
```

```
        // - x goes with column
        // - y goes with row
        const x = c * this.ts;
        const y = r * this.ts;

        // Draw the tile rectangle.
        rect(x, y, this.ts, this.ts);
      }
    }
  }
}

// ----------------------------
// 3) Hard-coded grid data (same as Example 1)
// ----------------------------

/*
This is identical in concept to Example 1: it's still "the level as a 2D array."
- We are not introducing new data structures here.
- We're showing that classes are mainly about organization + reuse.

The grid is structured as:
grid[row][col]
```

```
So grid[0] is the first row, grid[0][0] is the top-left cell.

*/

const grid = [

  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],

  [1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1],

  [1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1],

  [1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1],

  [1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1],

  [1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1],

  [1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1],

  [1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1],

  [1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1],

  [1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1],

  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],

]; // based on your Example 1 grid


// ----------------------------

// 4) Sketch state

// ----------------------------


// We'll store our Level instance here so draw() can access it.

let level;
```

```
// ----------------------------
// 5) p5.js lifecycle functions
// ----------------------------

function setup() {
  /*
  Create a Level instance from the grid.

  Think of this like:
  - “Here is some raw data (grid)”
  - “Wrap it in a Level object that knows what to do with it”
  */
  level = new Level(grid, TS);

  // Make the canvas exactly fit the level size.
  // This is a nice practical reason to have pixelWidth/pixelHeight helpers.
  createCanvas(level.pixelWidth(), level.pixelHeight());

  // Setup drawing styles.
  noStroke(); // cleaner tiles (no outlines) [file:11]
  textFont("sans-serif");
  textSize(14);
```

```
}

function draw() {
  // Clear the screen each frame.
  background(240);

  // Delegate drawing the maze to the level object.
  // This is the key structural difference from Example 1.
  level.draw();

  // Draw a small label so students can tell examples apart.
  fill(0);
  text("Level class → grid render", 10, 16);
}
```

Chat:

**/*
Week 4 — Example 2: Level class (hard-coded grid)
UPGRADE: Playable puzzle maze + 2 levels + collectibles + goal tile**

Controls:

- Arrow keys or WASD to move

- R to reset current level

```
Goal:
- Collect all circles, then reach the green exit.
*/

// ----------------------------
// 1) Global constants
// ----------------------------
const TS = 32;

// Tile legend:
// 0 = floor
// 1 = wall
// 2 = exit (goal)

// ----------------------------
// 2) Level class definition
// ----------------------------
class Level {
  constructor(grid, tileSize) {
    this.grid = grid;
    this.ts = tileSize;

    // Collectibles: auto-generate on floor tiles (0), but not on the player spawn
    this.collectibles = [];
    this._buildCollectibles();
  }
```

```
cols() { return this.grid[0].length; }
rows() { return this.grid.length; }
pixelWidth() { return this.cols() * this.ts; }
pixelHeight() { return this.rows() * this.ts; }

inBounds(r, c) {
  return r >= 0 && r < this.rows() && c >= 0 && c < this.cols();
}

tileAt(r, c) {
  if (!this.inBounds(r, c)) return 1; // treat out-of-bounds as wall
  return this.grid[r][c];
}

isWall(r, c) {
  return this.tileAt(r, c) === 1;
}

isExit(r, c) {
  return this.tileAt(r, c) === 2;
}

// Rebuild collectibles for this level
_buildCollectibles() {
  this.collectibles = [];
```

```
  for (let r = 0; r < this.rows(); r++) {
    for (let c = 0; c < this.cols(); c++) {
      const t = this.grid[r][c];
      if (t === 0) {
        this.collectibles.push({ r, c, eaten: false });
      }
    }
  }
}

// Remove collectible at a given tile (if exists)
eatAt(r, c) {
  for (let i = 0; i < this.collectibles.length; i++) {
    const p = this.collectibles[i];
    if (!p.eaten && p.r === r && p.c === c) {
      p.eaten = true;
      return true;
    }
  }
  return false;
}

remaining() {
  let left = 0;
  for (const p of this.collectibles) if (!p.eaten) left++;
  return left;
```

```
}

// Exclude a specific tile from having a collectible (player spawn)
clearCollectibleAt(r, c) {
  for (const p of this.collectibles) {
    if (p.r === r && p.c === c) p.eaten = true;
  }
}

draw() {
  for (let r = 0; r < this.rows(); r++) {
    for (let c = 0; c < this.cols(); c++) {
      const tileValue = this.grid[r][c];

      if (tileValue === 1) {
        fill(30, 50, 60); // wall
      } else if (tileValue === 2) {
        fill(80, 180, 110); // exit
      } else {
        fill(230); // floor
      }

      const x = c * this.ts;
      const y = r * this.ts;
      rect(x, y, this.ts, this.ts);
    }
```

```
    }

    // Draw collectibles (circles)
    for (const p of this.collectibles) {
      if (p.eaten) continue;
      const cx = p.c * this.ts + this.ts / 2;
      const cy = p.r * this.ts + this.ts / 2;
      fill(255, 160, 70);
      circle(cx, cy, this.ts * 0.45);
    }
  }
}

// ----------------------------
// 3) Two hard-coded grids
// ----------------------------

// Level 1: original maze with an added exit tile (2)
const grid1 = [
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
  [1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1],
  [1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1],
  [1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1],
  [1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1],
  [1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1],
  [1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1],
```

```
  [1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1],
  [1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1],
  [1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 2, 1], // exit at (9,14)
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
];

// Level 2: new maze layout, also with an exit tile (2)
const grid2 = [
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
  [1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 2, 1], // exit at (1,14)
  [1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1],
  [1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1],
  [1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1],
  [1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
  [1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1],
  [1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1],
  [1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1],
  [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1],
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
];

// ----------------------------
// 4) Game state
// ----------------------------
let levels = [];
let levelIndex = 0;
```

```
let level;

// Player is grid-based (tile coords)
let player = { r: 1, c: 1 };

// Game state
let gameWon = false;

// ----------------------------
// 5) Helpers
// ----------------------------
function loadLevel(i) {
  levelIndex = i;
  level = levels[levelIndex];

  // Spawn points (choose safe floor tiles)
  if (levelIndex === 0) {
    player.r = 1; player.c = 1;
  } else {
    player.r = 9; player.c = 1;
  }

  // Reset collectibles for the level
  level._buildCollectibles();
  level.clearCollectibleAt(player.r, player.c);
```

```
  gameWon = false;

  // Canvas size can change if levels differ (same size here, but keep correct)
  resizeCanvas(level.pixelWidth(), level.pixelHeight());
}

function tryMove(dr, dc) {
  if (gameWon) return;

  const nr = player.r + dr;
  const nc = player.c + dc;

  if (level.isWall(nr, nc)) return;

  player.r = nr;
  player.c = nc;

  // Eat collectible if present
  level.eatAt(player.r, player.c);

  // Win condition: all eaten + standing on exit
  if (level.remaining() === 0 && level.isExit(player.r, player.c)) {
    // If level 1 finished -> load level 2
    if (levelIndex < levels.length - 1) {
      loadLevel(levelIndex + 1);
    } else {
```

```
    gameWon = true;
   }
  }
}

// ----------------------------
// 6) p5.js lifecycle
// ----------------------------
function setup() {
  levels = [
   new Level(grid1, TS),
   new Level(grid2, TS),
  ];

  // Create canvas using first level size
  createCanvas(levels[0].pixelWidth(), levels[0].pixelHeight());

  noStroke();
  textFont("sans-serif");
  textSize(14);

  loadLevel(0);
}

function draw() {
  background(240);
```

```
level.draw();

// Draw player
const px = player.c * TS + TS / 2;
const py = player.r * TS + TS / 2;
fill(70, 120, 255);
circle(px, py, TS * 0.6);

// HUD
fill(0);
rect(0, 0, width, 28);
fill(255);
text(`Level: ${levelIndex + 1}/2   Circles left: ${level.remaining()}   (R = reset)`, 10, 18);

// Instructions + end screen
fill(0);
text("Collect all circles, then go to the green exit.", 10, height - 10);

if (gameWon) {
  fill(0, 180);
  rect(0, 0, width, height);
  fill(255);
  textSize(28);
  textAlign(CENTER, CENTER);
  text("YOU WIN", width / 2, height / 2);
```

```
   textAlign(LEFT, BASELINE);
   textSize(14);
  }
}

function keyPressed() {
  // Reset current level
  if (key === 'r' || key === 'R') {
   loadLevel(levelIndex);
   return;
  }

  // Arrow keys
  if (keyCode === LEFT_ARROW)  tryMove(0, -1);
  if (keyCode === RIGHT_ARROW) tryMove(0, 1);
  if (keyCode === UP_ARROW)    tryMove(-1, 0);
  if (keyCode === DOWN_ARROW)  tryMove(1, 0);

  // WASD
  if (key === 'a' || key === 'A') tryMove(0, -1);
  if (key === 'd' || key === 'D') tryMove(0, 1);
  if (key === 'w' || key === 'W') tryMove(-1, 0);
  if (key === 's' || key === 'S') tryMove(1, 0);
}
```

Ria:

Some of the circle you can't access because the maze is blocking it can you fix it

Chat:

```
// REPLACE your Level._buildCollectibles() with this reachable-only version,
// and add the helper method _reachableFrom(spawnR, spawnC).
// Then update loadLevel() to call the new builder.

// ----------------------------
// Inside class Level { ... }
// ----------------------------

// Flood-fill from spawn to find all reachable non-wall tiles
_reachableFrom(spawnR, spawnC) {
  const visited = Array.from({ length: this.rows() }, () =>
    Array(this.cols()).fill(false)
  );

  // If spawn is invalid, nothing reachable
  if (this.isWall(spawnR, spawnC)) return visited;

  const q = [];
  q.push({ r: spawnR, c: spawnC });
  visited[spawnR][spawnC] = true;

  const dirs = [
    { dr: -1, dc: 0 },
    { dr: 1, dc: 0 },
    { dr: 0, dc: -1 },
    { dr: 0, dc: 1 },
  ];

  while (q.length > 0) {
    const cur = q.shift();
    for (const d of dirs) {
      const nr = cur.r + d.dr;
      const nc = cur.c + d.dc;

      if (!this.inBounds(nr, nc)) continue;
      if (visited[nr][nc]) continue;
      if (this.isWall(nr, nc)) continue;

      visited[nr][nc] = true;
      q.push({ r: nr, c: nc });
    }
  }

  return visited;
}

// Build collectibles ONLY on reachable floor tiles (0), excluding spawn
_buildCollectiblesFromSpawn(spawnR, spawnC) {
  const reachable = this._reachableFrom(spawnR, spawnC);
  this.collectibles = [];

  for (let r = 0; r < this.rows(); r++) {
```

```
    for (let c = 0; c < this.cols(); c++) {
      if (!reachable[r][c]) continue;
      if (this.grid[r][c] !== 0) continue; // only floor gets circles
      if (r === spawnR && c === spawnC) continue;

      this.collectibles.push({ r, c, eaten: false });
    }
  }
}
// UPDATE your loadLevel(i) function to use the reachable-only collectible
builder.
// Replace the collectible reset lines with these.

function loadLevel(i) {
  levelIndex = i;
  level = levels[levelIndex];

  // Spawn points (choose safe floor tiles)
  if (levelIndex === 0) {
    player.r = 1; player.c = 1;
  } else {
    player.r = 9; player.c = 1;
  }

  // Rebuild collectibles ONLY where reachable
  level._buildCollectiblesFromSpawn(player.r, player.c);

  gameWon = false;

  resizeCanvas(level.pixelWidth(), level.pixelHeight());
}
```

Ria:

Code is not working now

Chat:

**/*

**PLAYABLE MAZE PUZZLE (2 LEVELS) — FIXED

**- Circles spawn ONLY on tiles reachable from the player spawn (no blocked circles)

**- Collect all circles, then reach the green exit to advance / win

**Controls: Arrow keys or WASD

**Reset: R

**/

```
const TS = 32;

// Tile legend:
// 0 = floor
// 1 = wall
// 2 = exit

class Level {
 constructor(grid, tileSize) {
  this.grid = grid;
  this.ts = tileSize;
  this.collectibles = [];
 }

 cols() { return this.grid[0].length; }
 rows() { return this.grid.length; }
 pixelWidth() { return this.cols() * this.ts; }
 pixelHeight() { return this.rows() * this.ts; }

 inBounds(r, c) {
  return r >= 0 && r < this.rows() && c >= 0 && c < this.cols();
 }

 tileAt(r, c) {
  if (!this.inBounds(r, c)) return 1;
```

```
  return this.grid[r][c];
}

isWall(r, c) { return this.tileAt(r, c) === 1; }
isExit(r, c) { return this.tileAt(r, c) === 2; }

// BFS flood-fill from spawn to mark all reachable non-wall tiles
_reachableFrom(spawnR, spawnC) {
  const visited = Array.from({ length: this.rows() }, () =>
    Array(this.cols()).fill(false)
  );

  if (this.isWall(spawnR, spawnC)) return visited;

  const q = [{ r: spawnR, c: spawnC }];
  visited[spawnR][spawnC] = true;

  const dirs = [
    { dr: -1, dc: 0 },
    { dr: 1, dc: 0 },
    { dr: 0, dc: -1 },
    { dr: 0, dc: 1 },
  ];

  while (q.length > 0) {
    const cur = q.shift();
```

```
    for (const d of dirs) {
      const nr = cur.r + d.dr;
      const nc = cur.c + d.dc;

      if (!this.inBounds(nr, nc)) continue;
      if (visited[nr][nc]) continue;
      if (this.isWall(nr, nc)) continue;

      visited[nr][nc] = true;
      q.push({ r: nr, c: nc });
    }
  }
  return visited;
}

// Build circles only on reachable floor tiles (0), excluding spawn tile
buildCollectiblesFromSpawn(spawnR, spawnC) {
  const reachable = this._reachableFrom(spawnR, spawnC);
  this.collectibles = [];

  for (let r = 0; r < this.rows(); r++) {
    for (let c = 0; c < this.cols(); c++) {
      if (!reachable[r][c]) continue;
      if (this.grid[r][c] !== 0) continue;
      if (r === spawnR && c === spawnC) continue;
```

```
      this.collectibles.push({ r, c, eaten: false });
    }
  }
}

eatAt(r, c) {
  for (let i = 0; i < this.collectibles.length; i++) {
    const p = this.collectibles[i];
    if (!p.eaten && p.r === r && p.c === c) {
      p.eaten = true;
      return true;
    }
  }
  return false;
}

remaining() {
  let left = 0;
  for (const p of this.collectibles) if (!p.eaten) left++;
  return left;
}

draw() {
  for (let r = 0; r < this.rows(); r++) {
    for (let c = 0; c < this.cols(); c++) {
      const tileValue = this.grid[r][c];
```

```
      if (tileValue === 1) fill(30, 50, 60);
      else if (tileValue === 2) fill(80, 180, 110);
      else fill(230);

      rect(c * this.ts, r * this.ts, this.ts, this.ts);
    }
   }

   for (const p of this.collectibles) {
    if (p.eaten) continue;
    fill(255, 160, 70);
    circle(p.c * this.ts + this.ts / 2, p.r * this.ts + this.ts / 2, this.ts * 0.45);
   }
  }
}

const grid1 = [
 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
 [1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1],
 [1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1],
 [1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1],
 [1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1],
 [1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1],
 [1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1],
 [1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1],
```

```
  [1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1],
  [1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 2, 1],
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
];

const grid2 = [
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
  [1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 2, 1],
  [1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1],
  [1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1],
  [1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1],
  [1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
  [1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1],
  [1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1],
  [1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1],
  [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1],
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
];

let levels = [];
let levelIndex = 0;
let level;

let player = { r: 1, c: 1 };
let gameWon = false;
```

```
function loadLevel(i) {
  levelIndex = i;
  level = levels[levelIndex];

  if (levelIndex === 0) {
    player.r = 1; player.c = 1;
  } else {
    player.r = 9; player.c = 1;
  }

  level.buildCollectiblesFromSpawn(player.r, player.c);
  gameWon = false;

  resizeCanvas(level.pixelWidth(), level.pixelHeight());
}

function tryMove(dr, dc) {
  if (gameWon) return;

  const nr = player.r + dr;
  const nc = player.c + dc;

  if (level.isWall(nr, nc)) return;

  player.r = nr;
  player.c = nc;
```

```
  level.eatAt(player.r, player.c);

  if (level.remaining() === 0 && level.isExit(player.r, player.c)) {
    if (levelIndex < levels.length - 1) loadLevel(levelIndex + 1);
    else gameWon = true;
  }
}

function setup() {
  levels = [new Level(grid1, TS), new Level(grid2, TS)];
  createCanvas(levels[0].pixelWidth(), levels[0].pixelHeight());

  noStroke();
  textFont("sans-serif");
  textSize(14);

  loadLevel(0);
}

function draw() {
  background(240);

  level.draw();

  // player
```

```
  fill(70, 120, 255);
  circle(player.c * TS + TS / 2, player.r * TS + TS / 2, TS * 0.6);

  // HUD
  fill(0);
  rect(0, 0, width, 28);
  fill(255);
  text(`Level: ${levelIndex + 1}/2   Circles left: ${level.remaining()}   (R = reset)`, 10, 18);

  fill(0);
  text("Collect all circles, then go to the green exit.", 10, height - 10);

  if (gameWon) {
    fill(0, 180);
    rect(0, 0, width, height);
    fill(255);
    textSize(28);
    textAlign(CENTER, CENTER);
    text("YOU WIN", width / 2, height / 2);
    textAlign(LEFT, BASELINE);
    textSize(14);
  }
}

function keyPressed() {
  if (key === 'r' || key === 'R') {
```

```
    loadLevel(levelIndex);
    return;
  }

  if (keyCode === LEFT_ARROW) tryMove(0, -1);
  if (keyCode === RIGHT_ARROW) tryMove(0, 1);
  if (keyCode === UP_ARROW) tryMove(-1, 0);
  if (keyCode === DOWN_ARROW) tryMove(1, 0);

  if (key === 'a' || key === 'A') tryMove(0, -1);
  if (key === 'd' || key === 'D') tryMove(0, 1);
  if (key === 'w' || key === 'W') tryMove(-1, 0);
  if (key === 's' || key === 'S') tryMove(1, 0);
}
```